

Bàn về ứng dụng của phản dịch hợp ngữ trong tối ưu hóa hằng số thời gian

Zhou Yisu
Trường Trung học số 7 Thành Đô, Tứ Xuyên

Nguồn gốc: On applications of disassembly in time-constant optimization

IOI China candidate team papers / 2006 / Zhou Yisu / Zhou Yisu.doc

Tóm tắt

Bài viết trình bày tầm quan trọng của việc tối ưu hóa hằng số thời gian. Trong môi trường Visual C++, từ mã hợp ngữ do một trình biên dịch cụ thể sinh ra, bài viết thảo luận cách dùng phản dịch hợp ngữ để phân tích hằng số thời gian và đề xuất một số phương án cải tiến.

Từ khóa: C++; phản dịch hợp ngữ; hằng số thời gian; tối ưu hóa.

1 Mở đầu

Việc tối ưu hóa chương trình là không có điểm dừng. Khi độ phức tạp tiệm cận giống nhau, hằng số thời gian là yếu tố then chốt quyết định chương trình chạy nhanh hay chậm. Tuy vậy trong thi lập trình, người ta thường tập trung vào độ phức tạp tiệm cận, còn vấn đề tối ưu hằng số thời gian, vốn cũng ảnh hưởng trực tiếp đến tốc độ, lại ít được coi trọng.

Bài viết này giải thích vì sao hằng số thời gian quan trọng, rồi trong môi trường Visual C++ khảo sát mã hợp ngữ do trình biên dịch sinh ra. Từ góc nhìn phản dịch, ta có thể nhận ra thao tác nào đắt, thao tác nào rẻ, và từ đó đổi cách viết chương trình để giảm hằng số.

2 Khái niệm cơ bản

Hợp ngữ là một ngôn ngữ lập trình cấp thấp gắn rất chặt với phần cứng. Nó có thể xem là dạng ký hiệu dễ nhớ và dễ hiểu hơn của mã máy. Do cú pháp hợp ngữ của các hệ khác nhau không tương thích với nhau, phần sau của bài viết lấy cú pháp Intel làm cơ sở và dùng Visual C++, một môi trường cũng dùng cú pháp Intel, làm nền tảng thử nghiệm.

Chương trình hợp ngữ được tạo từ các lệnh giả. Mỗi lệnh giả tương ứng với một chỉ thị mà bộ xử lý hiểu được. Chẳng hạn:

add

là lệnh cộng hai giá trị. Phần lớn lệnh giả có tham số:

add eax, edx

Lệnh này có hai tham số, một nguồn và một đích. Nó cộng giá trị nguồn vào giá trị đích rồi lưu kết quả tại đích. Tham số có nhiều loại, như thanh ghi, địa chỉ bộ nhớ, hoặc hằng số trực tiếp. Phần giới thiệu sau lấy thanh ghi làm ví dụ.

Thanh ghi có nhiều kích thước: 4 bit, 8 bit, 16 bit, 32 bit; với bộ xử lý MMX còn có thêm các loại khác. Trong chương trình 16 bit, ta chỉ có thể dùng thanh ghi 4 bit, 8 bit và 16 bit. Trong chương trình 32 bit, ta còn dùng được thanh ghi 32 bit.

Một số thanh ghi là một phần của thanh ghi khác. Ví dụ, nếu `eax` chứa giá trị `EA7823BBh`, thì các thanh ghi con `ax`, `ah` và `al` chứa các phần tương ứng:

Thanh ghi	Giá trị
<code>eax</code>	<code>EA 78 23 BB</code>
<code>ax</code>	<code>23 BB</code>
<code>ah</code>	<code>23</code>
<code>al</code>	<code>BB</code>

`ax`, `ah`, `al` đều là một phần của `eax`. `eax` là thanh ghi 32 bit, chỉ tồn tại từ 386 trở lên; `ax` chứa 16 bit thấp của `eax`; `ah` chứa byte cao của `ax`, còn `al` chứa byte thấp của `ax`. Vì thế `ax` là 16 bit, còn `ah` và `al` là 8 bit.

Xét đoạn mã sau:

```
mov eax, 12345678h
mov cl, ah
sub cl, 10
mov al, cl
```

Lệnh `mov` có thể chuyển một giá trị từ thanh ghi, bộ nhớ hoặc hằng số trực tiếp vào một thanh ghi khác. Ở ví dụ trên, `eax` nhận `12345678h`; sau đó giá trị của `ah`, tức byte thứ ba tính từ trái trong `eax`, được sao chép vào `cl`, byte thấp của thanh ghi `ecx`. Tiếp theo `cl` bị trừ đi 10 theo hệ thập phân và được chuyển về `al`, byte thấp của `eax`. Các kiến thức cơ bản khác về lệnh hợp ngữ có thể tham khảo trong các tài liệu nhập môn.

Trình biên dịch là chương trình dịch mã nguồn viết bằng ngôn ngữ cấp cao, thuận tiện cho con người viết, đọc và bảo trì, thành ngôn ngữ máy cấp thấp mà máy tính có thể nhận biết và chạy. Đầu vào của trình biên dịch là chương trình nguồn; đầu ra là chương trình tương đương trong ngôn ngữ đích. Chương trình nguồn thường viết bằng ngôn ngữ cấp cao như Pascal hoặc C++, trong khi chương trình đích là mã đối tượng, đôi khi cũng gọi là mã máy.

Thân thiện với trình biên dịch nghĩa là mã được viết phù hợp với quy tắc của trình biên dịch, tính đúng đắn của chức năng không phụ thuộc vào đặc tính riêng hoặc tính năng ẩn của một trình biên dịch cụ thể. Nhờ đó mã có tính phổ dụng và tương thích tốt hơn.

Kết luận của bài viết không nhất thiết yêu cầu cài đặt bằng hợp ngữ. Hợp ngữ nhưng không được mọi ngôn ngữ hỗ trợ, và dùng hợp ngữ nhưng cũng không phải ý định chính của bài viết.

3 Phân tích dữ liệu

Trên một nền tảng cụ thể, các lệnh hợp ngữ khác nhau có tốc độ thực thi khác nhau. Tuy nhiên, tốc độ tương đối của từng loại thao tác thường khá ổn định. Chẳng hạn, lệnh nhân và chia gần như chắc chắn chậm hơn nhiều so với lệnh cộng và trừ.

Bản gốc dẫn bằng thời gian lý thuyết của nhiều lệnh trên hệ 8286 từ tài liệu *The Art of Assembly Language*. Dù bảng số liệu ấy không được trích xuất đầy đủ từ tập Word, kết luận dùng trong bài vẫn là

thứ tự tương đối giữa các nhóm lệnh: di chuyển dữ liệu và logic rẽ, dịch bit và nhảy đắt hơn, nhân và đặc biệt là chia đắt hơn rất nhiều.

Để kiểm chứng nhận xét trên và khảo sát khả năng cải tiến chương trình từ góc nhìn phản dịch, tác giả thu thập số liệu chạy thực tế.

Môi trường thử nghiệm:

- Phần mềm: Windows XP SP2 050301-1519, MSVC++ 69514-335-0000007-18650.
- Phần cứng: Intel Pentium 4 CPU 2.66 GHz.

Trong bài, chế độ Debug tương đương chế độ không tối ưu, tức biên dịch không kèm tham số tối ưu. Chế độ Release tương đương chế độ tối ưu, gần với -O2 của G++. Một số kết luận không nhất thiết đúng với bộ xử lý, nền tảng hoặc trình biên dịch khác; khi dùng trong môi trường khác cần đo lại.

Trong phần đo tốc độ, thí nghiệm dùng mẫu C++ sau:

```
#include <time.h>
#include <stdio.h>

clock_t start, finish;
int a;

int main()
{
    start = clock();
    __asm
    {
        mov ecx, 2000000000
testBegin:
        mov eax, 10
        mov ebx, 10
        cdq
        /* insert the tested statement here */
        dec ecx
        jz testEnd
        jmp testBegin
testEnd:
    }
    finish = clock();
    printf("%.3lf", double(finish - start) / CLOCKS_PER_SEC);
    return 0;
}
```

Tác giả đo các thao tác được liệt kê trong tập `Configure.in` ở phụ lục bản gốc. Chạy script Python kèm theo cho kết quả trong bảng sau.

Thao tác thử nghiệm	Thời gian TB	Thời gian tương đối	Tỷ lệ
EMPTY	2.29	0.00	0.0
NORMAL IMUL	3.81	1.52	11.0
NORMAL IDIV	25.94	23.65	171.2
NORMAL TEST	2.43	0.14	1.0
NORMAL MOV	2.29	0.00	0.0
NORMAL ADD	2.41	0.12	0.9
NORMAL INC	3.05	0.76	5.5
NORMAL SUB	2.41	0.12	0.9
NORMAL DEC	3.05	0.76	5.5
NORMAL XOR	2.35	0.06	0.4
NORMAL AND	2.35	0.06	0.4
NORMAL SHL	2.44	0.15	1.1
NORMAL SAR	2.44	0.15	1.1
NORMAL SHR	2.44	0.16	1.1
NORMAL NOT	2.29	0.00	0.0
NORMAL XCHG	3.04	0.76	5.5
PTR MOV	2.28	0.00	0.0
PTR ADD	3.05	0.76	5.5
PTR SUB	3.05	0.76	5.5
PTR XOR	3.05	0.76	5.5
PTR AND	3.05	0.76	5.5
PTR XCHG	67.39	65.10	471.2
JMP	3.14	0.85	6.2
PUSH POP	3.05	0.76	5.5
CALL RET JMP	9.16	6.88	49.8
EMPTY (verify)	2.29	0.00	0.0

Cần lưu ý rằng kết quả đo có sai số và dao động theo hệ điều hành, trình biên dịch và cấu hình hệ thống. Hơn nữa bộ xử lý hiện đại không thực thi chương trình theo đúng mô hình “từng câu lệnh một” đơn giản: còn có bộ đệm giải mã, bộ nhớ đệm L1, L2, độ dài toán hạng 16/32 bit và nhiều yếu tố khác. Vì vậy số liệu trên chỉ nên coi là xấp xỉ thực nghiệm.

Dựa vào kết quả này, tác giả chia hằng số thời gian của các nhóm lệnh thành sáu mức:

Nhóm thao tác	Tỷ lệ thời gian
mov, lea; di chuyển dữ liệu; and, or, xor, not; add, sub; test	1
shl, shr, sal, sar; dịch bit	1.5–2
Lấy giá trị qua con trỏ; push+pop; jmp	4
mul, imul; nhân	5
div, idiv; chia	25
call+ret; gọi hàm rồi trở về	27

Từ các mức này ta thấy độ nhanh chậm tương đối của mã hợp ngữ. Nếu từ góc độ hợp ngữ, ta xóa bỏ được một thao tác có tỷ lệ thời gian lớn hoặc thay nó bằng tổ hợp các thao tác có tỷ lệ nhỏ, thì hằng số thời gian của chương trình đã được tối ưu.

4 Phân tích ví dụ

4.1 Một thí nghiệm nhỏ về memset

Ta biết memset là thao tác có độ phức tạp $O(N)$. Trước hết xét chương trình C++ sau, giả sử máy tính có đủ bộ nhớ:

```
#include <string.h>
```

```
const int Total = 1000000000;
const int Time = /* a legal value you like */;
```

```

char field[Total / Time];
int i, j;

int main()
{
    for (j = 0; j < 10; j++)
        for (i = 0; i < Time; i++)
            memset(field, 0, sizeof(field));
    return 0;
}

```

Đừng vội chạy chương trình. Có thể trả lời ngay quan hệ giữa giá trị Time và tốc độ chạy không? Có thể ta sẽ nghĩ Time không ảnh hưởng đến tốc độ, vì tổng lượng dữ liệu được gán vẫn giữ nguyên. Nhưng khi thử trên máy, ta thấy chương trình nhanh nhất khi Time vào khoảng 100000; nếu Time nhỏ hơn hoặc lớn hơn nhiều thì tốc độ đều giảm rõ rệt.

Khi Time nhỏ, chương trình chậm đi có thể giải thích bằng chi phí cấp phát và quản lý bộ nhớ của Windows. Khi Time lớn, khác biệt giữa hằng số thời gian của vòng lặp và của chính thao tác memset làm thời gian tăng lên.

Từ góc độ phản dịch, lệnh `rep stos` rất nhanh: mỗi lần thực thi xấp xỉ chỉ chiếm một đơn vị thời gian nhưng vừa hoàn thành thao tác ghi giá trị vừa giảm biến đếm vòng lặp. Trong khi đó một vòng lặp viết ở mức C++ phải gán biến và nhảy, xấp xỉ chiếm 14 đơn vị thời gian mỗi vòng. Vì hằng số khác nhau như vậy, khi Time lớn, tức số lần gọi tăng lên, chương trình chạy chậm hơn.

Vì sao Debug và Release có hiệu suất khác nhau ở phía giá trị Time lớn? Trong C++, ta gọi `memset` như một hàm. Thực ra hệ thống có lệnh hợp ngữ chuyên biệt để làm cùng việc, chẳng hạn `rep stos`. Ở chế độ Release, trình biên dịch tối ưu `memset` thành mã hợp ngữ dạng sau:

```

memset(table, 0, sizeof(table));
00401001 xor      eax, eax
00401003 mov     ecx, 9C40h
00401008 mov     edi, offset table (4072C8h)
0040100D rep stos dword ptr [edi]

```

Ở chế độ Debug, tuy lõi của `memset` cũng là `rep stos`, nhưng vì `memset` được gọi như một hàm nên hệ thống dùng tổ hợp `push`, `call`, `ret`; chi phí thực tế cao hơn:

```

memset(field, 0, sizeof(field));
00411A6B push    2710h
00411A70 push    0
00411A72 push    offset field (4284E8h)
00411A77 call    @ILT+350(_memset) (411163h)
00411A7C add     esp, 0Ch

```

Vì quá trình thực hiện `memset` trong Debug tốn hằng số lớn hơn, đường thời gian chạy của Debug cao hơn Release khi Time lớn. Qua hiện tượng các cách cài đặt `memset` có hiệu suất khác nhau, ta thấy tối ưu chương trình là khả thi ngay cả khi độ phức tạp tiệm cận không đổi.

4.2 Tối ưu hằng số của lời gọi hàm

Hằng số gọi hàm là chi phí phát sinh bởi các lệnh như `push`, `pop`, `call`, `ret` trong quá trình gọi hàm. Quá trình này khác nhau rất rõ giữa Debug và Release.

Trong Debug, ta thường dùng từ khóa `inline` để tối ưu mã. Nhưng `inline` chỉ là gợi ý cho trình biên dịch, không phải lệnh bắt buộc, ít nhất là trong môi trường Visual Studio. Trong môi trường thì dùng GCC không kèm tham số, `inline` có tính thỏa hiệp hơn: nếu dùng thì thường được biên dịch thành hàm nội tuyến, nếu không thì thành hàm thường.

Hàm thử nghiệm:

```
inline void swap(int& a, int& b)
{
    int t = a;
    a = b;
    b = t;
}
```

Mã gọi trong Debug:

```
swap(a, b);
004133AD lea    eax, [b]
004133B0 push   eax
004133B1 lea    ecx, [a]
004133B4 push   ecx
004133B5 call   swap (41158Ch)
004133BA add     esp, 8
```

Trình biên dịch còn đi qua một bảng nhảy:

```
0041157D jmp    _aligned_free_dbg (41B4D0h)
00411582 jmp    _RTC_GetErrorFunc (416C30h)
00411587 jmp    GetStringTypeW (425788h)
0041158C jmp    swap (411B20h)
00411591 int    3
00411592 int    3
```

Do đó hằng số của chương trình tăng vì các lệnh `call` và `ret` phụ. Hơn nữa để thuận tiện đặt điểm dừng khi gỡ lỗi, Microsoft tạo bảng nhảy trong bộ nhớ ở chế độ Debug, làm hằng số tiếp tục tăng.

Điều này gợi ý một ví dụ quen thuộc: cách trình biên dịch thực hiện các hàm trong thư viện STL. Mã C++ của STL dùng rất nhiều kế thừa và nạp chồng, vì thế ngay cả một thao tác đơn giản cũng có thể đi qua nhiều tầng gọi khi xem bằng chế độ phản dịch. Với các đoạn chạy trong vòng lặp sâu, ta nên cân nhắc vấn đề này. Tác giả nêu hai phương án thay thế.

Không dùng hàm con. Đây là cách trực tiếp nhất: không có hàm con thì không có chi phí gọi hàm. Tuy nhiên chương trình sẽ dài hơn và khó gỡ lỗi hơn, nên chỉ nên dùng có chọn lọc.

Dùng macro. Macro có nhiều hạn chế vì chỉ là khai triển cơ học. Nhưng với một số chức năng đơn giản và được gọi rất thường xuyên, dùng macro có thể giảm chi phí lời gọi. Mặc dù không khuyến khích định nghĩa biến bên trong khối `define`, vì có thể gây lỗi khó lường, ta có thể dùng biến toàn cục để giải quyết cùng vấn đề. Ví dụ sau xử lý biến tạm của `swap` theo cách đó:

```
int tmp;
#define swap(A, B) tmp = A, A = B, B = tmp
```

```
int main()
{
    int a = 3, b = 4;
    swap(a, b);
    return 0;
}
```

Nhưng macro cũng có nhược điểm lớn. Đoạn mã sau cho thấy thể nào là khai triển cơ học:

```
#include <stdio.h>
#include <stdlib.h>

int getRand()
{
    int _t = rand();
    printf("Call getRand return %d\n", _t);
    return _t;
}

#define max(A, B) ((A) > (B) ? (A) : (B))

int main()
{
    srand(543210);
    printf("Get the max value: %d\n", max(getRand(), getRand()));
    return 0;
}
```

Kết quả chạy ổn định:

```
Call getRand return 4461
Call getRand return 14545
Call getRand return 3994
Get the max value: 3994
```

Do đối số được khai triển nhiều lần, `getRand()` bị gọi ba lần thay vì hai lần. Vì vậy macro chỉ thích hợp với các biểu thức không có tác dụng phụ, hoặc với những tình huống ta kiểm soát chặt được cách dùng.

Trong Release, mọi hàm đều được ưu tiên thử biên dịch như hàm nội tuyến, nên việc viết rõ từ khóa `inline` thường không còn tác dụng thực tế. Với hàm:

```
void swap(int& a, int& b)
{
    int t = a;
    a = b;
    b = t;
}
```

dù không có `inline`, trong mã phản dịch đã không còn thấy lời gọi `swap`:

```

a++;
0040105B add  eax, 1
swap(a, b);
0040105E mov  dword ptr [esp+8], eax
a *= b;
printf("%d%d\n", a, i);
return 0;
00401062 imul eax, ecx

```

Đây không phải tin xấu. Trong STL, gần như mọi hàm đều không viết kèm inline, nhưng ở chế độ Release chúng vẫn thường được biên dịch thành nội tuyến. Nhờ đó chi phí call+ret giảm đi, và đây là một nguyên nhân khiến hiệu suất STL trong Debug và Release khác nhau rất lớn.

4.3 Tối ưu phép chia và phép lấy dư

Phép lấy dư $c = a \% b$ tương đương với

$$c = a - a/b,$$

nhưng trong hiện thực bên trong, nó dùng trực tiếp giá trị trả về thứ hai của phép chia:

```

a %= b;
00411B53 mov  eax, dword ptr [a]
00411B56 cdq
00411B57 idiv  eax, dword ptr [b]
00411B5A mov  dword ptr [a], edx

```

Vì vậy tốc độ của phép lấy dư gần bằng phép chia tương ứng, và cách tối ưu cũng tương tự.

Theo kết quả đo ở trên, idiv là lệnh có tỷ lệ thời gian rất lớn. Các nhà thiết kế trình biên dịch cũng biết điều này, nên trong đa số trường hợp trình biên dịch có thể chuyển phép chia cho hằng số thành phép dịch bit nhanh hơn nhiều. Chẳng hạn, với $a \neq 2$, trong đó a là số nguyên 32 bit, chế độ Debug dịch thành:

```

a /= 2;
00411B4F mov  eax, dword ptr [a]
00411B52 cdq
00411B53 sub  eax, edx
00411B55 sar  eax, 1
00411B57 mov  dword ptr [a], eax

```

Cách này nhanh hơn nhiều so với thực hiện idiv. Tuy vậy phép nhân và chia phải xử lý thêm các trường hợp đặc biệt như dấu âm và tràn số; điều này biểu hiện trực tiếp thành mã hợp ngữ thừa. Nếu một phép toán có thể được thay trực tiếp bằng dịch bit, ta nên cân nhắc dùng dịch bit.

Trí thông minh của trình biên dịch cũng có giới hạn. Khi một biến chia cho một biến khác, trình biên dịch không thể biết tính chất đặc biệt của biến chia, nên thường dịch thẳng thành idiv. Nếu số chia thật ra có dạng đặc biệt, tiềm năng tối ưu sẽ bị bỏ lỡ. Khi ta tự biết tính chất đó, có thể viết tối ưu thủ công. Ví dụ:

```

const a[] = {1, 2, 4, 8, 16, 32, 64, 128,
             256, 512, 1024, 2048, 4096};

```



```
c = b % a[i];
d = e / a[i];
```

có thể đổi thành:

```
const a[] = {1, 2, 4, 8, 16, 32, 64, 128,
             256, 512, 1024, 2048, 4096};
```

```
c = b & (a[i] - 1);
d = e >> (i - 1);
```

Điều kiện đúng ở đây là các phần tử của a đều là lũy thừa của hai. Trong cài đặt thực tế cần đặc biệt chú ý chỉ số và dấu của toán hạng.

4.4 Tối ưu hiệu suất của mảng nhiều chiều

Mảng là cấu trúc dữ liệu gần như mọi ngôn ngữ cấp cao đều có. Nó gom nhiều dữ liệu vào một biến; từng phần tử được tham chiếu bằng một chỉ số với mảng một chiều, hoặc nhiều chỉ số với mảng lồng nhau và mảng nhiều chiều. Việc gom dữ liệu vào mảng giúp quản lý dữ liệu đơn giản hơn, chẳng hạn khi truyền một tập dữ liệu cho hàm chỉ cần dùng một định danh.

Trong bộ nhớ, mảng nhiều chiều được trải phẳng. Ví dụ một mảng hai chiều 4×4

```
(0,0) (0,1) (0,2) (0,3)
(1,0) (1,1) (1,2) (1,3)
(2,0) (2,1) (2,2) (2,3)
(3,0) (3,1) (3,2) (3,3)
```

được lưu liên tiếp theo hàng:

```
(0,0) (0,1) (0,2) (0,3)
(1,0) (1,1) (1,2) (1,3)
(2,0) (2,1) (2,2) (2,3)
(3,0) (3,1) (3,2) (3,3)
```

Vì cần chuyển đổi từ chỉ số nhiều chiều sang địa chỉ tuyến tính, phép định vị phần tử mảng nhiều chiều trong Debug thường được dịch thành phép nhân. Trong Release, trình biên dịch tối ưu theo cách tương tự tối ưu phép nhân.

Với mảng `a[10][10]`, thao tác lấy giá trị trong Debug dùng `imul`:

```
return a[i][j];
00411B6F mov    eax, dword ptr [i]
00411B75 imul   eax, eax, 28h
00411B78 lea    ecx, a[eax]
00411B7F mov    edx, dword ptr [j]
00411B85 mov    eax, dword ptr [ecx+edx*4]
```

Trong Release, thao tác này được tối ưu thành `lea` và định địa chỉ có chỉ số, gần giống tối ưu phép nhân:

```
return a[i][j];
```

```

0040102E  mov  eax, dword ptr [esp+18h]
00401032  lea   edx, [eax+eax*4]
00401035  mov  eax, dword ptr [esp+14h]
00401039  add  esp, 0Ch
0040103C  lea   ecx, [eax+edx*2]
0040103F  mov  eax, dword ptr [esp+ecx*4+10h]

```

Vì `imul` là lệnh có tỷ lệ thời gian lớn, nếu ta loại bỏ được lệnh này trong phép tính địa chỉ thì có thể cải thiện tốc độ đáng kể. Từ đó sinh ra một cách tối ưu khá khéo: nếu cách thay đổi chỉ số là cố định, chẳng hạn trong tìm kiếm theo chiều rộng trên lưới với các bước $+1, -1, +N, -N$, thì dùng con trỏ `type*` và phép cộng trừ con trỏ cùng biến phụ có thể nhanh hơn, vì bỏ được phép nhân ẩn.

Thí nghiệm sau minh họa điều đó. Với mảng `d[10][10]`, dùng toán tử `[]` để lấy giá trị:

```

a = d[i][j];
00411B55  mov  eax, dword ptr [i (42B694h)]
00411B5A  imul  eax, eax, 28h
00411B5D  mov  ecx, dword ptr [j (42B690h)]
00411B63  mov  edx, dword ptr [eax+ecx*4+42B500h]
00411B6A  mov  dword ptr [a (42B6A0h)], edx

```

Dùng con trỏ để lấy giá trị:

```

b = *dd;
00411B8B  mov  eax, dword ptr [dd (42B69Ch)]
00411B90  mov  ecx, dword ptr [eax]
00411B92  mov  dword ptr [b (42B698h)], ecx

```

Phép trượt con trỏ có thể cài bằng một mảng hằng số hoặc bằng nhiều đoạn mã. Ví dụ:

```

int table[200][200];
int *ptr, *ptr2;

/* East, South, West, North */
const go[] = {1, 200, -1, -200};

```

Khi sử dụng:

```

ptr2 = ptr + go[0];
00411A4C  mov  eax, dword ptr [go (42401Ch)]
00411A51  mov  ecx, dword ptr [ptr (44F5E8h)]
00411A57  lea  edx, [ecx+eax*4]
00411A5A  mov  dword ptr [ptr2 (4284E0h)], edx
return *ptr2;
00411A60  mov  eax, dword ptr [ptr2 (4284E0h)]
00411A65  mov  eax, dword ptr [eax]

```

Như vậy phép nhân ẩn ban đầu đã bị loại bỏ. Tác giả gọi kỹ thuật này là thao tác “đi bộ” của con trỏ. Điều kiện dùng được kỹ thuật này là cách biến đổi con trỏ phải cố định.

Ví dụ: adv1900 của NOI 2005. Trong một ma trận $N \times M$ có một số ô chứa ngai và một vật nằm ở một ô nào đó. Vật sẽ di chuyển theo lịch: trong mỗi giây, nó có một hướng quy định và một đơn vị di chuyển ứng với một ô. Ở mỗi giây, ta có thể cho vật di chuyển hoặc đứng yên. Hỏi vật có thể di chuyển được độ dài lớn nhất là bao nhiêu.

Lịch gồm nhiều đoạn thời gian; trong mỗi đoạn, vật luôn di chuyển theo cùng một hướng. Quy mô dữ liệu:

- 50% dữ liệu: $1 \leq N, M \leq 200$, thời gian $T \leq 200$.
- 100% dữ liệu: $1 \leq N, M \leq 200$, số đoạn thời gian $K \leq 200$, thời gian $T \leq 40000$.

Bài này có nhiều cách giải; cách tối ưu dùng tính đơn điệu để giảm chiều. Dù dùng phương pháp nào, trong quá trình giải đều phải qua một bước then chốt: chuyển trạng thái giữa các thời điểm khác nhau. Trong lời giải tối ưu, bước này được xử lý theo lô, một lần thao tác hoàn thành chuyển tiếp cho cả một đoạn thời gian. Các tối ưu khác như heap hoặc RMQ cũng dựa trên tư tưởng xử lý theo lô.

Tuy nhiên, sau các thí nghiệm ở trên, ta có thể đi theo một hướng khác. Bước chuyển trạng thái của bài này có đúng đặc điểm rất thích hợp để tối ưu hằng số: nó nằm trong vòng lặp trong cùng và ảnh hưởng trực tiếp đến tốc độ; đồng thời nó dùng rất nhiều thao tác trên mảng với cách biến đổi chỉ số cố định, có thể thay bằng con trỏ “đi bộ”.

Bảng dưới ghi kết quả trước và sau khi dùng tối ưu này. Đơn vị thời gian là giây.

Chương trình	0	1	2	3	4	5	6	7	8	9	Tổng
Không đi bộ	0.016	0.016	0.016	0.063	0.344	1.016	0.781	1.031	0.625	1.156	5.064
Có đi bộ	0.016	0.016	0.016	0.047	0.234	0.703	0.547	0.734	0.484	0.797	3.594

So với phương pháp chuẩn, vốn có thể đạt khoảng 0.3 giây ở điểm lớn nhất, cách này vẫn còn khoảng cách lớn. Nhưng nó cho phép một phương pháp “sơ cấp” mà gần như ai cũng nghĩ ra vượt qua toàn bộ dữ liệu trong giới hạn thời gian.

5 Tổng kết

Bài viết khảo sát hằng số thời gian của nhiều thao tác từ hai phía: mã hợp ngữ do một trình biên dịch C++ cụ thể sinh ra và tốc độ tương đối của các nhóm lệnh hợp ngữ. Qua một số ví dụ, bài viết phân tích nguyên nhân ảnh hưởng đến hằng số thời gian từ góc nhìn phản dịch, đồng thời đề xuất các cách tối ưu tương ứng.

Hợp ngữ là ngôn ngữ lập trình gần với bản chất của máy tính nhất. Vì tính đặc thù đó, nó cũng là một trong những ngôn ngữ khó nắm vững và khó gỡ lỗi nhất. Nhưng nếu hiểu và dùng tốt, tính nhanh, hiệu quả và gọn của nó là một công cụ rất mạnh. Ngôn ngữ cấp cao xét đến cùng cũng được hiện thực bằng hợp ngữ. Vì thế trong bối cảnh yêu cầu hiệu suất ngày càng nghiêm ngặt, hiểu “tập tính” của ngôn ngữ là điều rất quan trọng.

Sau khi nắm được đặc điểm của hợp ngữ, dù trong lập trình thông thường hay trong thi đấu, việc vận dụng đầy đủ các đặc tính ấy có thể nâng tốc độ chương trình thêm một bậc, từ đó giải quyết bài toán thực tế nhanh hơn và tốt hơn.

Lời cảm ơn

Trước hết tác giả cảm ơn thầy Zhang Junliang đã tận tình hướng dẫn trong nhiều năm và dành nhiều tâm huyết cho sự trưởng thành của tác giả; cảm ơn thầy Chen Wenyu của Đại học Khoa học Kỹ thuật Điện tử

đã sửa bản thảo và góp ý; cảm ơn giáo viên chủ nhiệm Zhang Jiancheng đã quan tâm và ủng hộ; cảm ơn các bạn Min Kerui, Bai Yunfeng, Yang Shuang và Wang Kai vì nhiều trao đổi hữu ích.

Tài liệu tham khảo

1. *The Art of Assembly Language*.
2. MSDN China, phần Visual C++.
3. National Olympiad in Informatics 2005.

Phụ lục

Bản gốc có các tệp phụ lục gồm thư viện đo tốc độ lệnh xấp xỉ, một phép đo khác sinh bởi quy trình tương tự, phần hiện thực hợp ngữ của memset trong Visual Studio, và chương trình adv1900 trước và sau tối ưu con trỏ “đi bộ”. Nội dung trích xuất được từ tập Word chỉ giữ lại các đoạn mã và bảng kết quả đã đưa vào các phần liên quan ở trên; các danh sách chương trình dài không được chép lại vì không phải trọng tâm của bản dịch.